

CS 171 / EE 142: Intro to ML and DM

Christian Shelton

UC Riverside

Slide Set 16: Neural Networks



- From UC Riverside
 - ▶ CS 171 / EE 142: Introduction to Machine Learning and Data Mining
 - ▶ Professor Christian Shelton
- DO NOT REDISTRIBUTE
 - ▶ For use only by enrolled students in the course

Neural Networks, initial notation

If $g(\cdot)$ is a scalar-to-scalar function,

let $\tilde{g}(\cdot)$ be a vector-to-vector function which is the element-wise application of g to a vector.

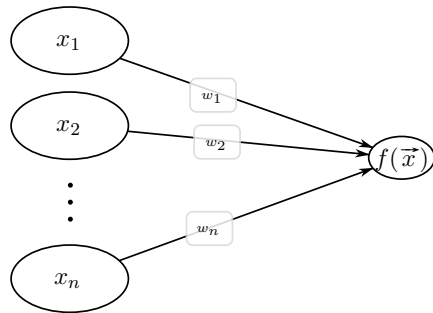
Examples:

$$g(z) = z^2 \quad \Rightarrow \quad \tilde{g}\left(\begin{bmatrix} z_1 \\ z_2 \\ \vdots \\ z_n \end{bmatrix}\right) = \begin{bmatrix} z_1^2 \\ z_2^2 \\ \vdots \\ z_n^2 \end{bmatrix}$$

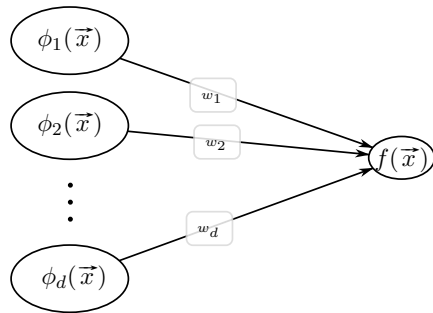
$$g(z) = \log(z) \quad \Rightarrow \quad \tilde{g}\left(\begin{bmatrix} z_1 \\ z_2 \\ \vdots \\ z_n \end{bmatrix}\right) = \begin{bmatrix} \log(z_1) \\ \log(z_2) \\ \vdots \\ \log(z_n) \end{bmatrix}$$

$$\vec{w}^\top \vec{x} = f(\vec{x})$$

$$\vec{w}^\top \vec{x} = f(\vec{x})$$

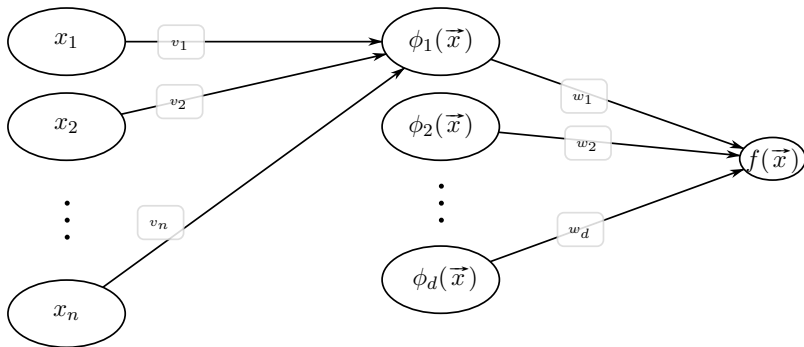


$$\vec{w}^\top \phi(\vec{x}) = f(\vec{x})$$



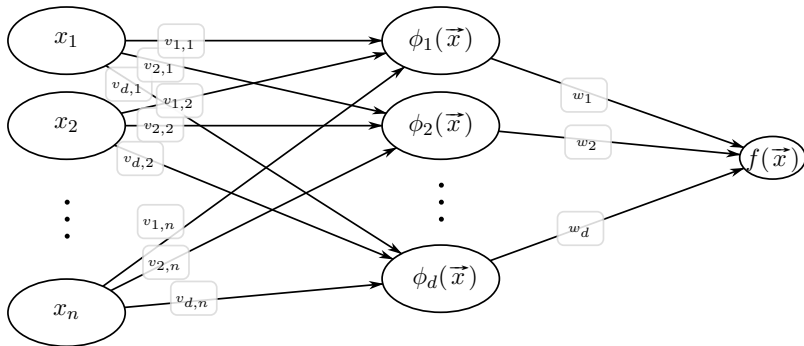
$$\vec{v}^\top \vec{x} = \phi_1(\vec{x})$$

$$\vec{w}^\top \phi(\vec{x}) = f(\vec{x})$$



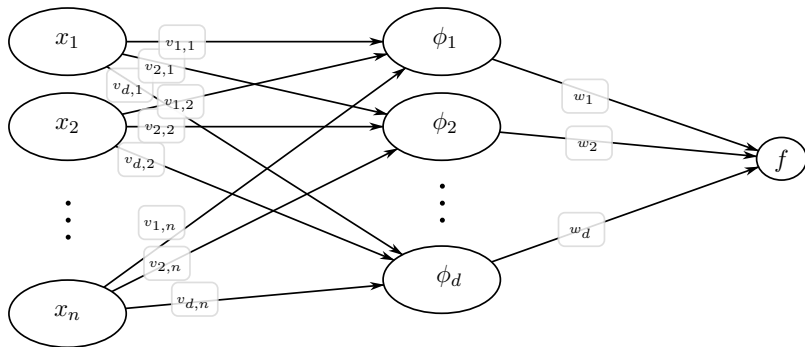
$$\mathbf{V}\vec{x} = \phi(\vec{x})$$

$$\vec{w}^\top \phi(\vec{x}) = f(\vec{x})$$



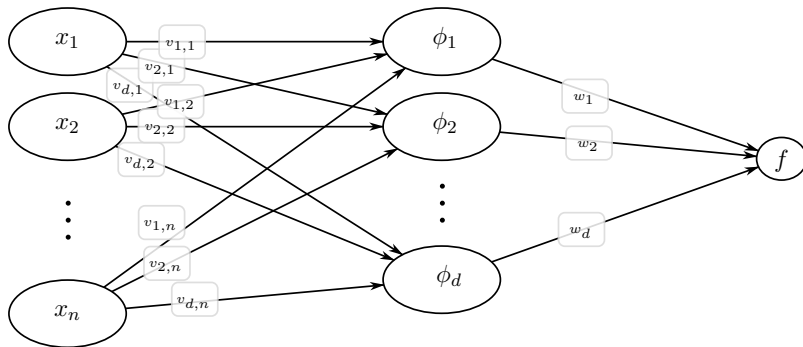
$$\mathbf{V}\vec{x} = \vec{\phi}$$

$$\vec{w}^\top \vec{\phi} = f$$



$$\tilde{g}(\mathbf{V}\vec{x}) = \vec{\phi}$$

$$\vec{w}^\top \vec{\phi} = f$$

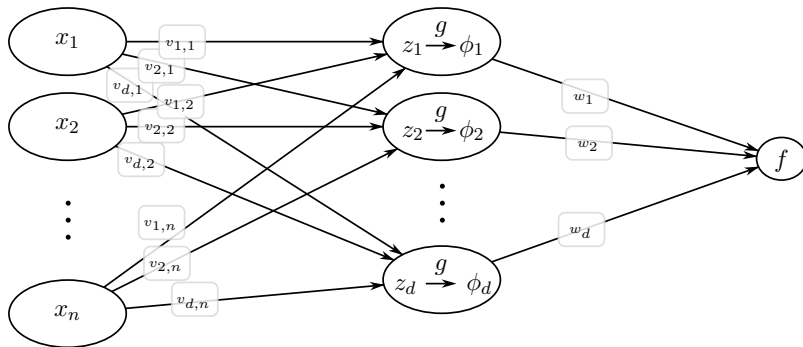


Neural Networks

$$V\vec{x} = \vec{z}$$

$$\tilde{g}(\vec{z}) = \vec{\phi}$$

$$\vec{w}^\top \vec{\phi} = f$$



Neural Networks

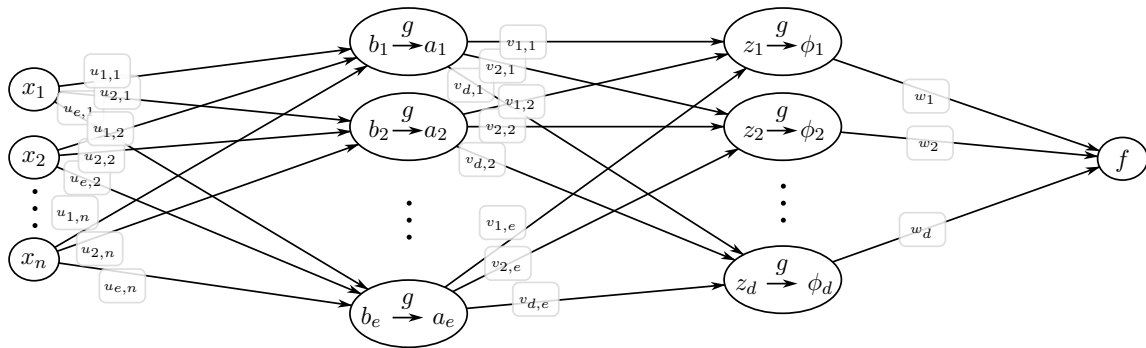
$$U\vec{x} = \vec{b}$$

$$\tilde{g}(\vec{b}) = \vec{a}$$

$$V\vec{a} = \vec{z}$$

$$\tilde{g}(\vec{z}) = \vec{\phi}$$

$$\vec{w}^\top \vec{\phi} = f$$



Neural Networks

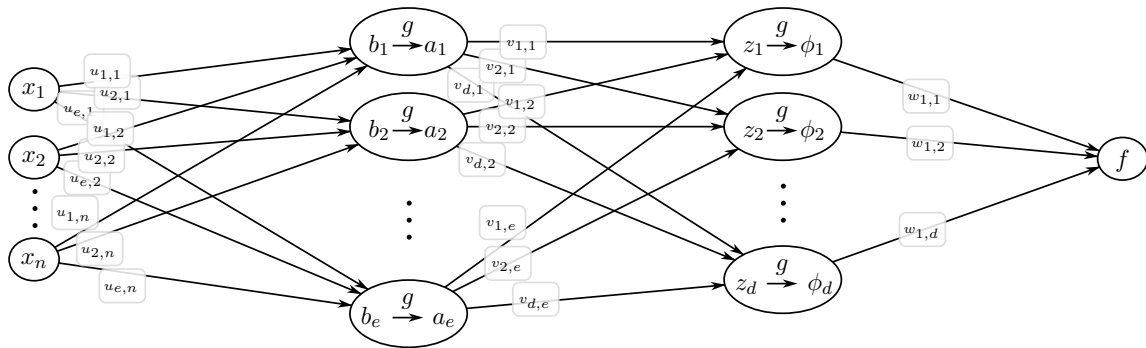
$$U\vec{x} = \vec{b}$$

$$\tilde{g}(\vec{b}) = \vec{a}$$

$$V\vec{a} = \vec{z}$$

$$\tilde{g}(\vec{z}) = \vec{\phi}$$

$$W\vec{\phi} = f$$



Neural Networks

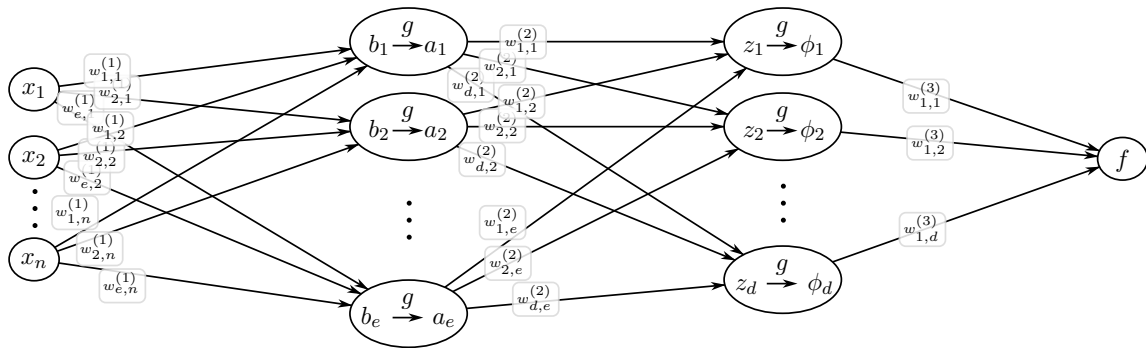
$$W^{(1)} \vec{x} = \vec{b}$$

$$\tilde{g}(\vec{b}) = \vec{a}$$

$$W^{(2)} \vec{a} = \vec{z}$$

$$\tilde{g}(\vec{z}) = \vec{\phi}$$

$$W^{(3)} \vec{\phi} = f$$



Neural Networks

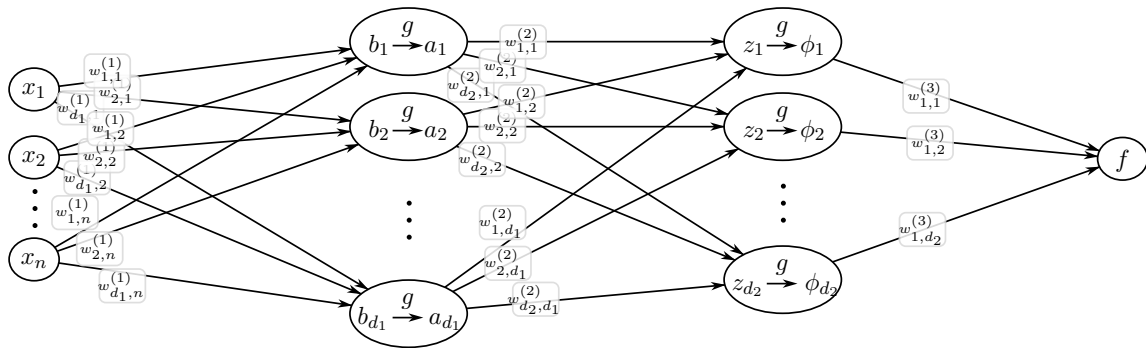
$$W^{(1)} \vec{x} = \vec{b}$$

$$\tilde{g}(\vec{b}) = \vec{a}$$

$$W^{(2)} \vec{a} = \vec{z}$$

$$\tilde{g}(\vec{z}) = \vec{\phi}$$

$$W^{(3)} \vec{\phi} = f$$



Neural Networks

$$W^{(1)} \vec{a}^{(0)} = \vec{z}^{(1)}$$

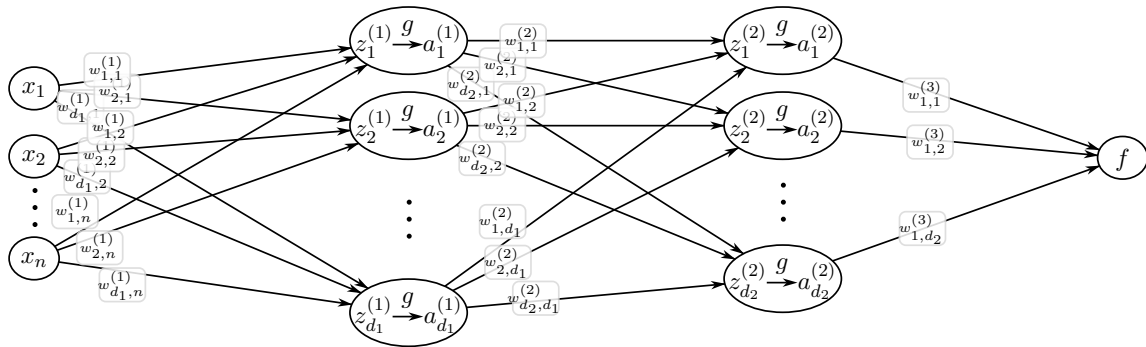
$$W^{(2)} \vec{a}^{(1)} = \vec{z}^{(2)}$$

$$W^{(3)} \vec{a}^{(2)} = f$$

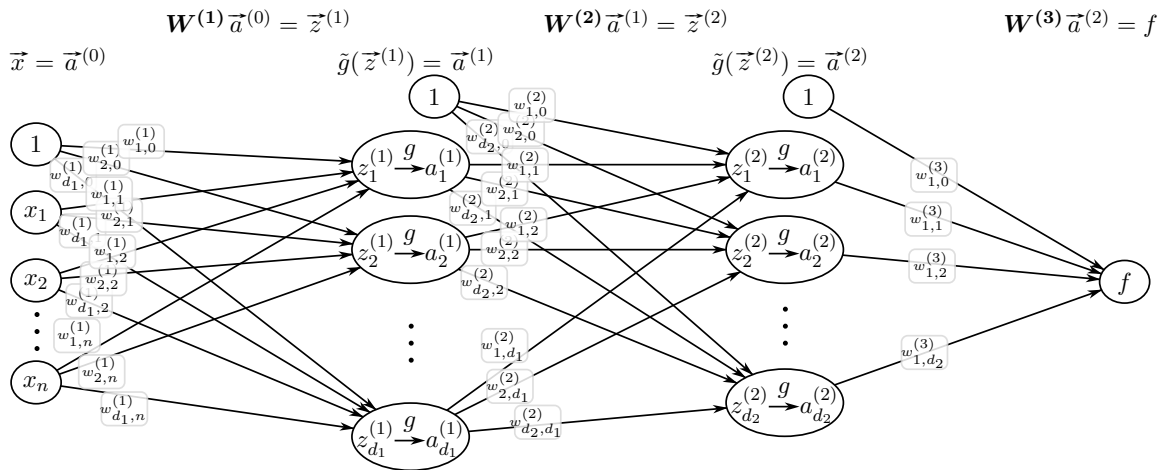
$$\vec{x} = \vec{a}^{(0)}$$

$$\tilde{g}(\vec{z}^{(1)}) = \vec{a}^{(1)}$$

$$\tilde{g}(\vec{z}^{(2)}) = \vec{a}^{(2)}$$



Neural Networks



Neural Networks

$$f = \mathbf{W}^{(3)} \tilde{g}(\mathbf{W}^{(2)} \tilde{g}(\mathbf{W}^{(1)} \vec{x}))$$

$$\mathbf{W}^{(1)} \vec{a}^{(0)} = \vec{z}^{(1)}$$

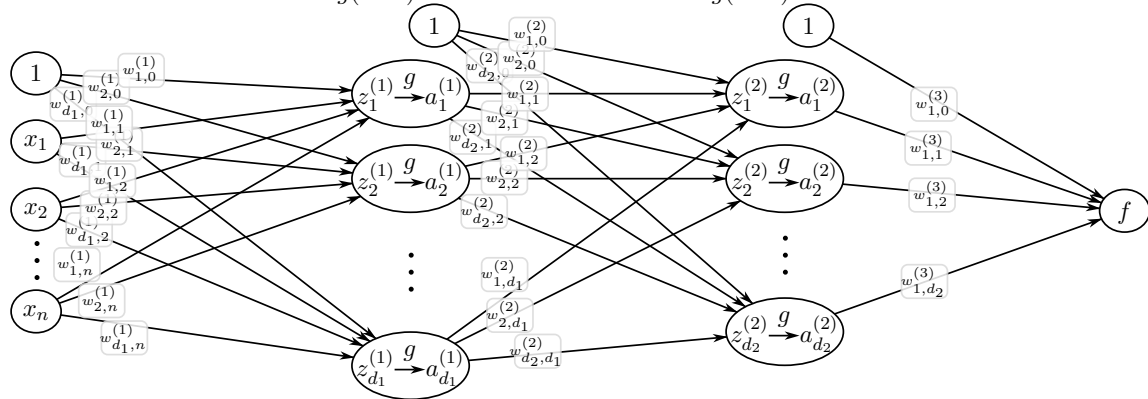
$$\mathbf{W}^{(2)} \vec{a}^{(1)} = \vec{z}^{(2)}$$

$$\mathbf{W}^{(3)} \vec{a}^{(2)} = f$$

$$\vec{x} = \vec{a}^{(0)}$$

$$\tilde{g}(\vec{z}^{(1)}) = \vec{a}^{(1)}$$

$$\tilde{g}(\vec{z}^{(2)}) = \vec{a}^{(2)}$$



Neural Networks

3 layer neural network

$$W^{(1)} \vec{a}^{(0)} = \vec{z}^{(1)}$$

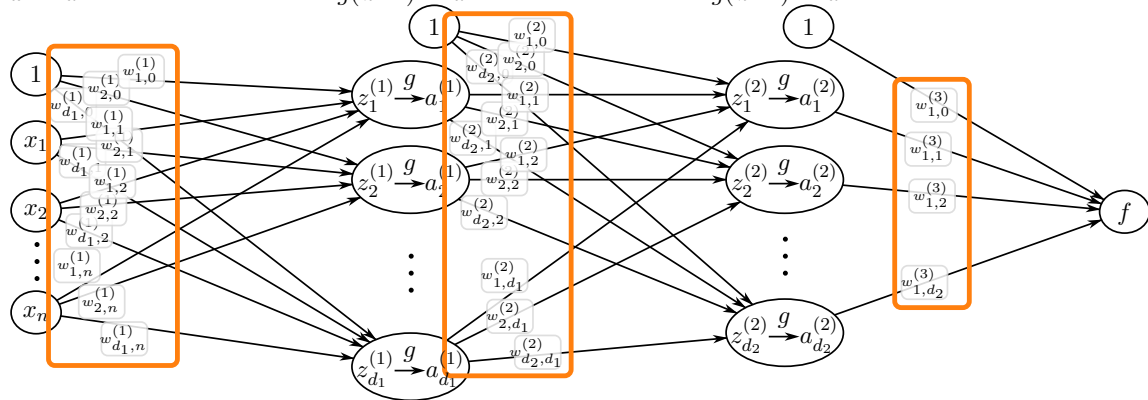
$$W^{(2)} \vec{a}^{(1)} = \vec{z}^{(2)}$$

$$W^{(3)} \vec{a}^{(2)} = f$$

$$\vec{x} = \vec{a}^{(0)}$$

$$\tilde{g}(\vec{z}^{(1)}) = \vec{a}^{(1)}$$

$$\tilde{g}(\vec{z}^{(2)}) = \vec{a}^{(2)}$$



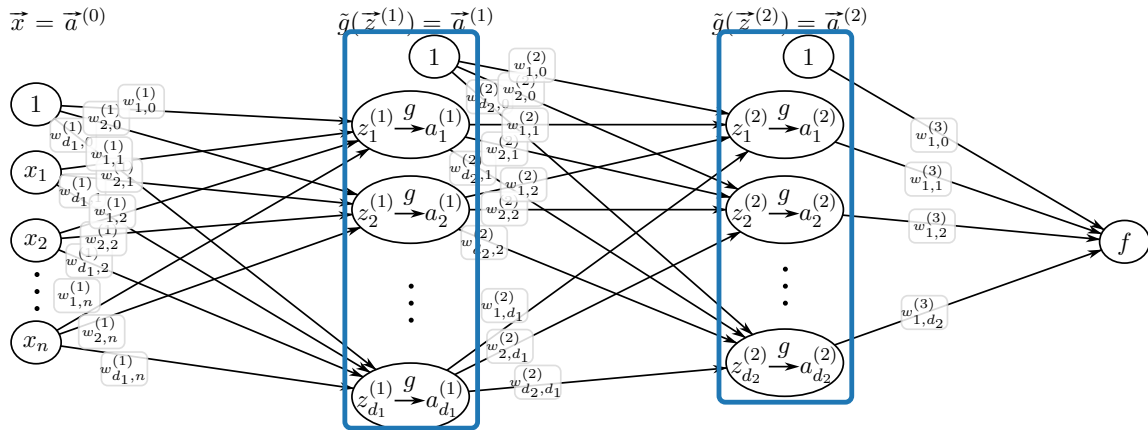
Neural Networks

2 hidden layer neural network

$$W^{(1)} \vec{a}^{(0)} = \vec{z}^{(1)}$$

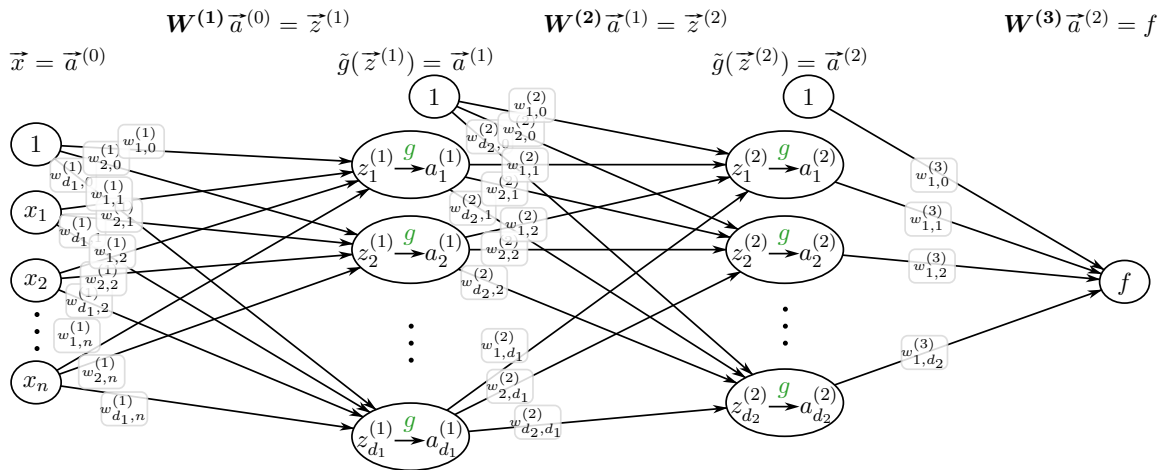
$$W^{(2)} \vec{a}^{(1)} = \vec{z}^{(2)}$$

$$W^{(3)} \vec{a}^{(2)} = f$$



Neural Networks

non-linearity / activation function: g



Neural Networks

activation(s): a

$$W^{(1)} \vec{a}^{(0)} = \vec{z}^{(1)}$$

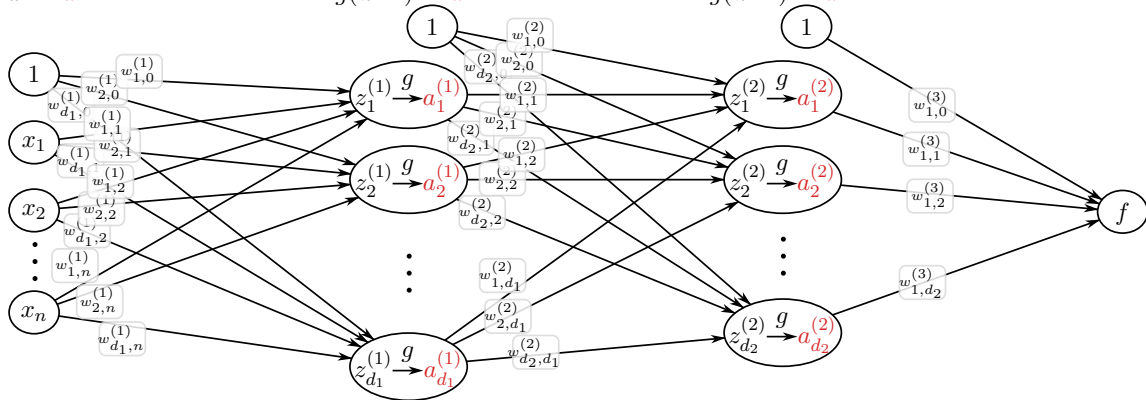
$$W^{(2)} \vec{a}^{(1)} = \vec{z}^{(2)}$$

$$W^{(3)} \vec{a}^{(2)} = f$$

$$\vec{x} = \vec{a}^{(0)}$$

$$\tilde{g}(\vec{z}^{(1)}) = \vec{a}^{(1)}$$

$$\tilde{g}(\vec{z}^{(2)}) = \vec{a}^{(2)}$$



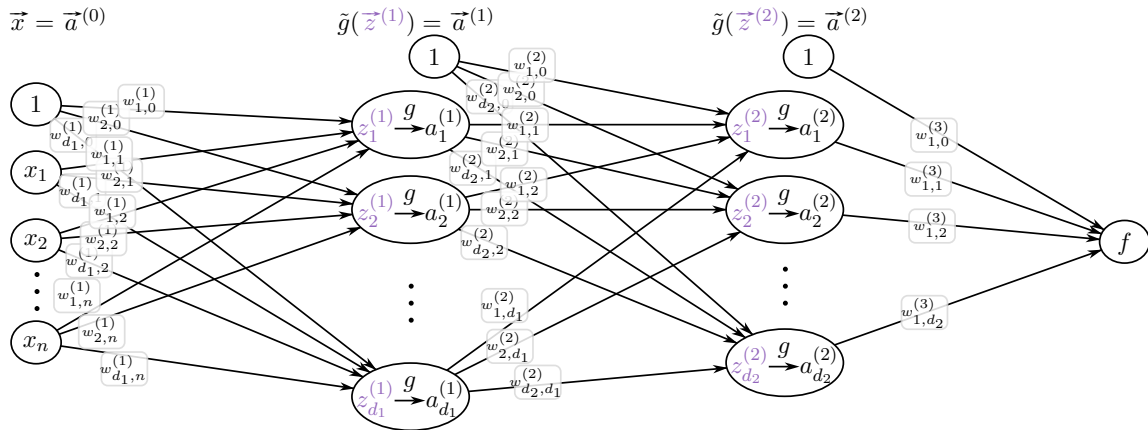
Neural Networks

pre-activation(s): z

$$W^{(1)} \vec{a}^{(0)} = \vec{z}^{(1)}$$

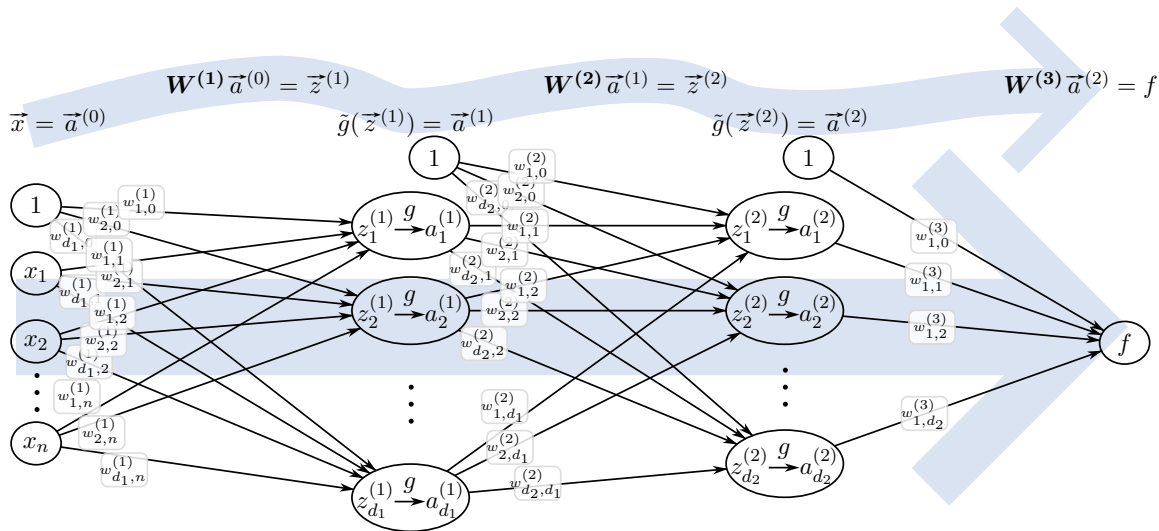
$$W^{(2)} \vec{a}^{(1)} = \vec{z}^{(2)}$$

$$W^{(3)} \vec{a}^{(2)} = f$$



Neural Networks

forward propagation (calculates $f(\vec{x})$ from $\vec{x}$)



Notes on Neural Networks

- A neural network is just a (more complicated) functional form for $f(\vec{x})$ or $\tilde{f}(\vec{x})$.

Notes on Neural Networks

- A neural network is just a (more complicated) functional form for $f(\vec{x})$ or $\tilde{f}(\vec{x})$.
- This is a layered neural network
 - ▶ Also known as a multi-layer perceptron
 - ▶ NOTE: term perceptron does not mean trained with the perceptron learning algorithm
 - ▶ Other more complicated architectures are possible

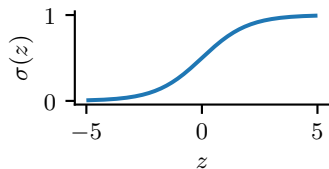
Common Non-linearities

- The activation function, g :
 - ▶ Can be different for each layer (or each unit), but usually is not
 - ▶ Can be almost anything, provided it is non-linear
 - ▶ Common choices are
 - ▶ $g(z) = \sigma(z)$ (not ideal)
 - ▶ $g(z) = \tanh(z)$ (better)
 - ▶ $g(z) = \max(0, z)$ (called a rectified linear unit, ReLU)

Common Non-linearities

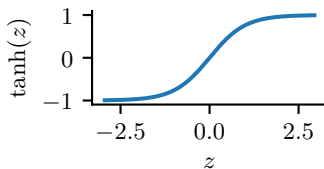
- The activation function, g :

- ▶ Can be different for each layer (or each unit), but usually is not
- ▶ Can be almost anything, provided it is non-linear
- ▶ Common choices are
 - ▶ $g(z) = \sigma(z)$ (not ideal)
 - ▶ $g(z) = \tanh(z)$ (better)
 - ▶ $g(z) = \max(0, z)$ (called a rectified linear unit, ReLU)



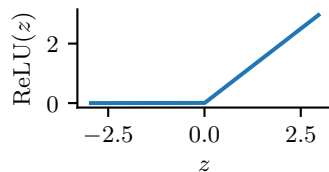
$$\sigma(z) = \frac{1}{1+e^{-z}}$$

$$\sigma'(z) = \sigma(z)(1 - \sigma(z))$$



$$\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$$

$$\tanh'(z) = 1 - \tanh^2(z)$$



$$\text{ReLU}(z) = \max(0, z)$$

$$\text{ReLU}'(z) = \begin{cases} 0 & \text{if } z < 0 \\ 1 & \text{otherwise} \end{cases}$$

Forward Propagation Example

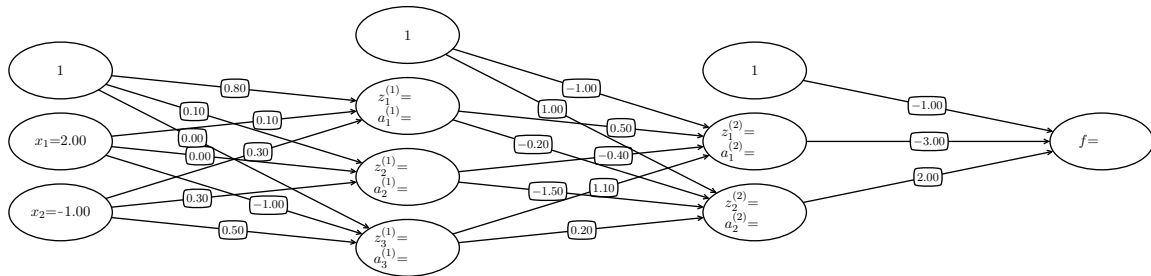
$$\mathbf{W}^{(1)} = \begin{bmatrix} 0.80 & 0.10 & 0.30 \\ 0.10 & 0.00 & 0.30 \\ 0.00 & -1.00 & 0.50 \end{bmatrix}$$

$$\vec{z}^{(1)} = \quad \vec{a}^{(1)} =$$

$$\mathbf{W}^{(2)} = \begin{bmatrix} -1.00 & 0.50 & -0.40 & 1.10 \\ 1.00 & -0.20 & -1.50 & 0.20 \end{bmatrix}$$

$$\vec{z}^{(2)} = \quad \vec{a}^{(2)} =$$

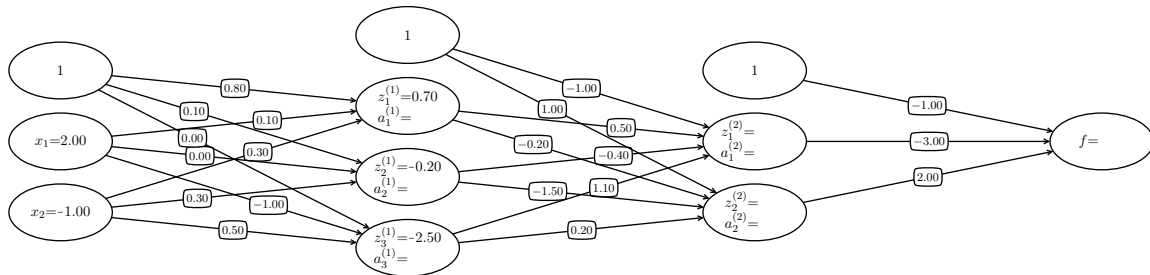
$$\mathbf{W}^{(3)} = \begin{bmatrix} -1.00 & -3.00 & 2.00 \end{bmatrix}$$



$$g(z) = \tanh(z)$$

Forward Propagation Example

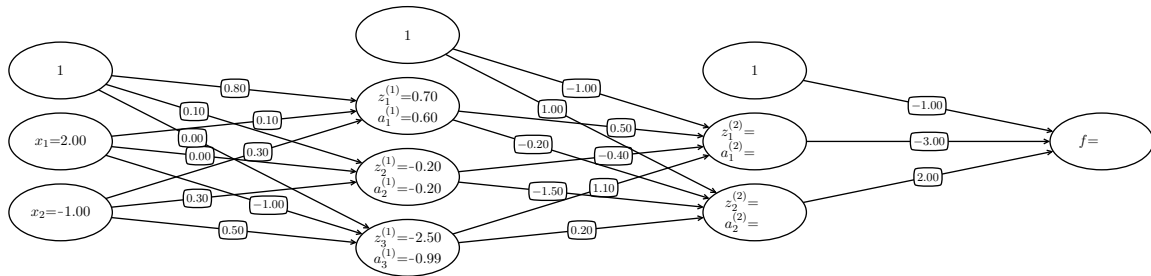
$$\mathbf{W}^{(1)} = \begin{bmatrix} 0.80 & 0.10 & 0.30 \\ 0.10 & 0.00 & 0.30 \\ 0.00 & -1.00 & 0.50 \end{bmatrix} \quad \vec{z}^{(1)} = \vec{a}^{(1)} = \begin{bmatrix} 0.70 \\ -0.20 \\ -2.50 \end{bmatrix} \quad \mathbf{W}^{(2)} = \begin{bmatrix} -1.00 & 0.50 & -0.40 & 1.10 \\ 1.00 & -0.20 & -1.50 & 0.20 \end{bmatrix} \quad \vec{z}^{(2)} = \vec{a}^{(2)} = \begin{bmatrix} -1.00 \\ -3.00 \\ 2.00 \end{bmatrix} \quad \mathbf{W}^{(3)} = \begin{bmatrix} -1.00 & -3.00 & 2.00 \end{bmatrix}$$



$$g(z) = \tanh(z)$$

Forward Propagation Example

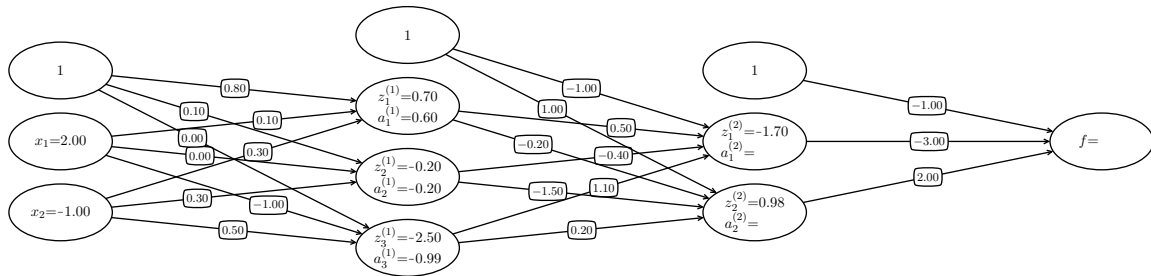
$$\mathbf{W}^{(1)} = \begin{bmatrix} 0.80 & 0.10 & 0.30 \\ 0.10 & 0.00 & 0.30 \\ 0.00 & -1.00 & 0.50 \end{bmatrix} \quad \vec{z}^{(1)} = \begin{bmatrix} 0.70 \\ -0.20 \\ -2.50 \end{bmatrix} \quad \vec{a}^{(1)} = \begin{bmatrix} 1.00 \\ 0.60 \\ -0.20 \\ -0.99 \end{bmatrix} \quad \mathbf{W}^{(2)} = \begin{bmatrix} -1.00 & 0.50 & -0.40 & 1.10 \\ 1.00 & -0.20 & -1.50 & 0.20 \end{bmatrix} \quad \vec{z}^{(2)} = \begin{bmatrix} -1.00 \\ 1.00 \\ -0.20 \\ -1.50 \end{bmatrix} \quad \vec{a}^{(2)} = \begin{bmatrix} 0.50 \\ -0.40 \\ 1.10 \\ 0.20 \end{bmatrix} \quad \mathbf{W}^{(3)} = \begin{bmatrix} -1.00 & -3.00 & 2.00 \end{bmatrix}$$



$$g(z) = \tanh(z)$$

Forward Propagation Example

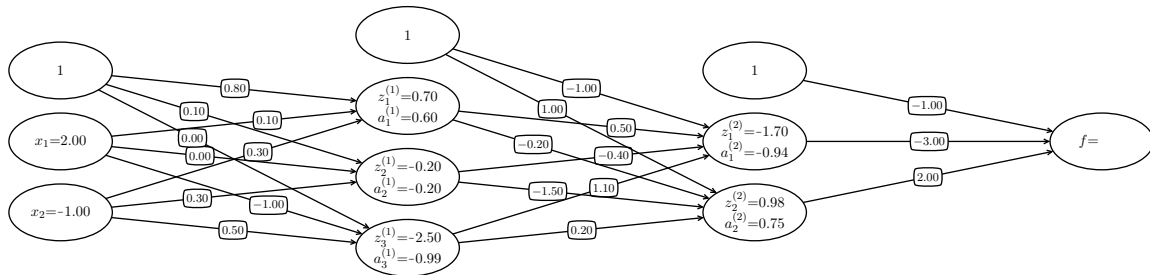
$$\mathbf{W}^{(1)} = \begin{bmatrix} 0.80 & 0.10 & 0.30 \\ 0.10 & 0.00 & 0.30 \\ 0.00 & -1.00 & 0.50 \end{bmatrix} \quad \vec{z}^{(1)} = \begin{bmatrix} 0.70 \\ -0.20 \\ -2.50 \end{bmatrix} \quad \vec{a}^{(1)} = \begin{bmatrix} 1.00 \\ 0.60 \\ -0.20 \\ -0.99 \end{bmatrix} \quad \mathbf{W}^{(2)} = \begin{bmatrix} -1.00 & 0.50 & -0.40 & 1.10 \\ 1.00 & -0.20 & -1.50 & 0.20 \end{bmatrix} \quad \vec{z}^{(2)} = \begin{bmatrix} -1.70 \\ 0.98 \end{bmatrix} \quad \vec{a}^{(2)} = \begin{bmatrix} -1.70 \\ 0.98 \end{bmatrix} \quad \mathbf{W}^{(3)} = \begin{bmatrix} -1.00 & -3.00 & 2.00 \end{bmatrix}$$



$$g(z) = \tanh(z)$$

Forward Propagation Example

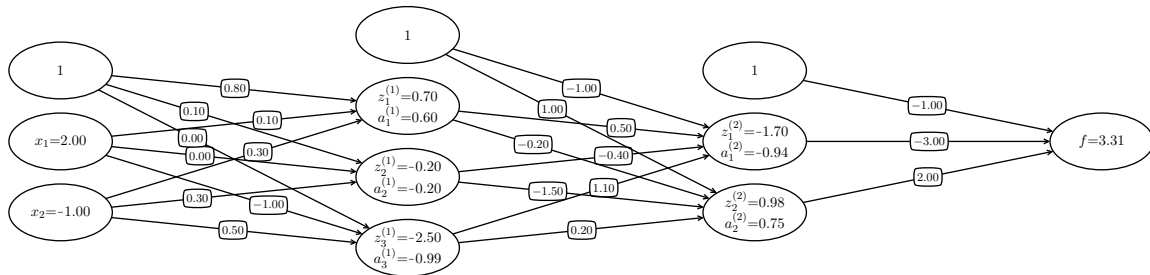
$$\mathbf{W}^{(1)} = \begin{bmatrix} 0.80 & 0.10 & 0.30 \\ 0.10 & 0.00 & 0.30 \\ 0.00 & -1.00 & 0.50 \end{bmatrix} \quad \vec{z}^{(1)} = \vec{a}^{(1)} = \begin{bmatrix} 1.00 \\ 0.60 \\ -0.20 \\ -2.50 \end{bmatrix} \quad \mathbf{W}^{(2)} = \begin{bmatrix} -1.00 & 0.50 & -0.40 & 1.10 \\ 1.00 & -0.20 & -1.50 & 0.20 \end{bmatrix} \quad \vec{z}^{(2)} = \vec{a}^{(2)} = \begin{bmatrix} 1.00 \\ -1.70 \\ -0.94 \\ 0.98 \end{bmatrix} \quad \mathbf{W}^{(3)} = \begin{bmatrix} -1.00 & -3.00 & 2.00 \end{bmatrix}$$



$$g(z) = \tanh(z)$$

Forward Propagation Example

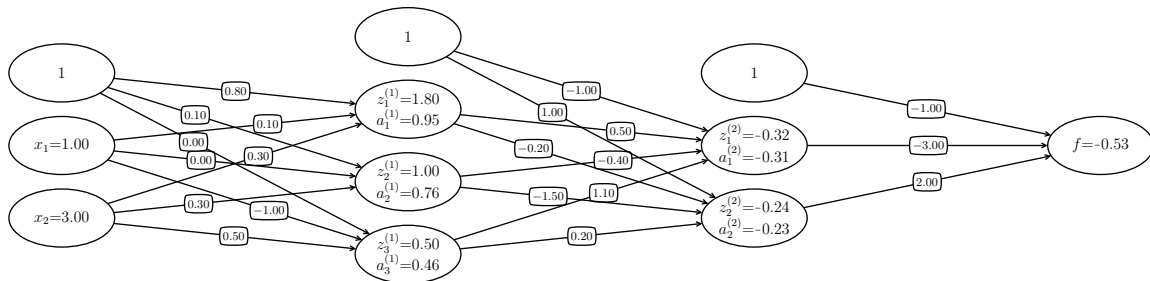
$$\mathbf{W}^{(1)} = \begin{bmatrix} 0.80 & 0.10 & 0.30 \\ 0.10 & 0.00 & 0.30 \\ 0.00 & -1.00 & 0.50 \end{bmatrix} \quad \vec{z}^{(1)} = \vec{a}^{(1)} = \begin{bmatrix} 1.00 \\ 0.60 \\ -0.20 \\ -2.50 \end{bmatrix} \quad \mathbf{W}^{(2)} = \begin{bmatrix} -1.00 & 0.50 & -0.40 & 1.10 \\ 1.00 & -0.20 & -1.50 & 0.20 \end{bmatrix} \quad \vec{z}^{(2)} = \vec{a}^{(2)} = \begin{bmatrix} 1.00 \\ -1.70 \\ -0.94 \\ 0.98 \end{bmatrix} \quad \mathbf{W}^{(3)} = \begin{bmatrix} -1.00 & -3.00 & 2.00 \end{bmatrix}$$



$$g(z) = \tanh(z)$$

Forward Propagation Example

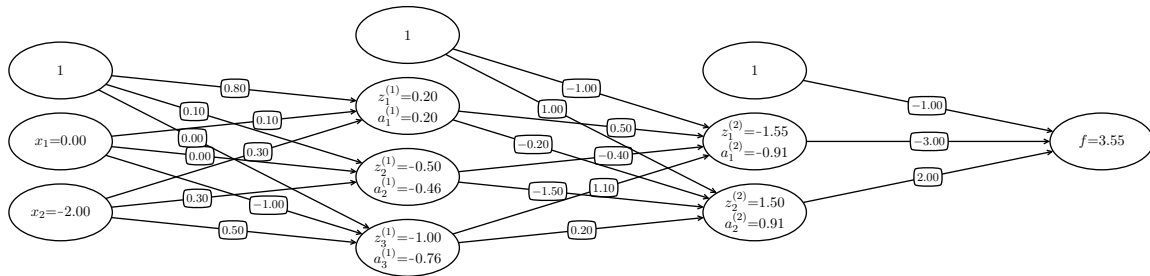
$$\mathbf{W}^{(1)} = \begin{bmatrix} 0.80 & 0.10 & 0.30 \\ 0.10 & 0.00 & 0.30 \\ 0.00 & -1.00 & 0.50 \end{bmatrix} \quad \vec{z}^{(1)} = \vec{a}^{(1)} = \begin{bmatrix} 1.80 \\ 1.00 \\ 0.50 \end{bmatrix} \quad \mathbf{W}^{(2)} = \begin{bmatrix} -1.00 & 0.50 & -0.40 & 1.10 \\ 1.00 & -0.20 & -1.50 & 0.20 \end{bmatrix} \quad \vec{z}^{(2)} = \vec{a}^{(2)} = \begin{bmatrix} -0.32 \\ -0.31 \\ -0.24 \end{bmatrix} \quad \mathbf{W}^{(3)} = \begin{bmatrix} -1.00 & -3.00 & 2.00 \end{bmatrix}$$



$$g(z) = \tanh(z)$$

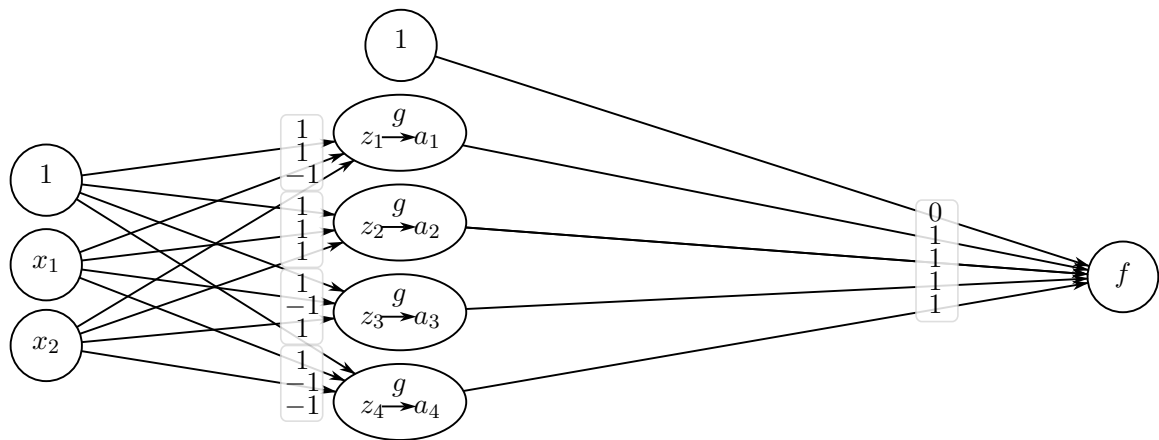
Forward Propagation Example

$$\mathbf{W}^{(1)} = \begin{bmatrix} 0.80 & 0.10 & 0.30 \\ 0.10 & 0.00 & 0.30 \\ 0.00 & -1.00 & 0.50 \end{bmatrix} \quad \vec{z}^{(1)} = \vec{a}^{(1)} = \begin{bmatrix} 1.00 \\ 0.20 \\ -0.50 \\ -1.00 \end{bmatrix} \quad \mathbf{W}^{(2)} = \begin{bmatrix} -1.00 & 0.50 & -0.40 & 1.10 \\ 1.00 & -0.20 & -1.50 & 0.20 \end{bmatrix} \quad \vec{z}^{(2)} = \vec{a}^{(2)} = \begin{bmatrix} 1.00 \\ -1.55 \\ -0.91 \\ 1.50 \end{bmatrix} \quad \mathbf{W}^{(3)} = \begin{bmatrix} -1.00 & -3.00 & 2.00 \end{bmatrix}$$

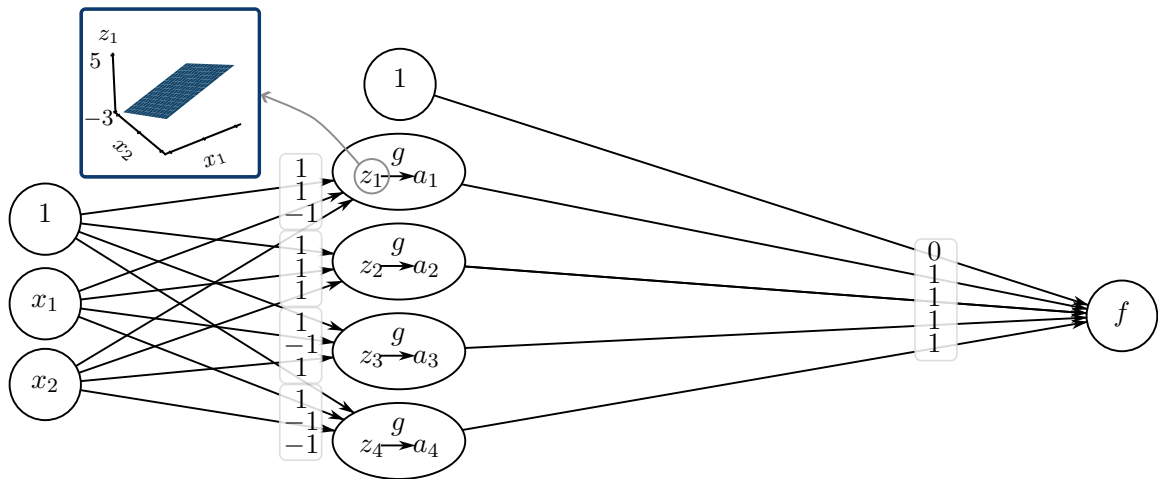


$$g(z) = \tanh(z)$$

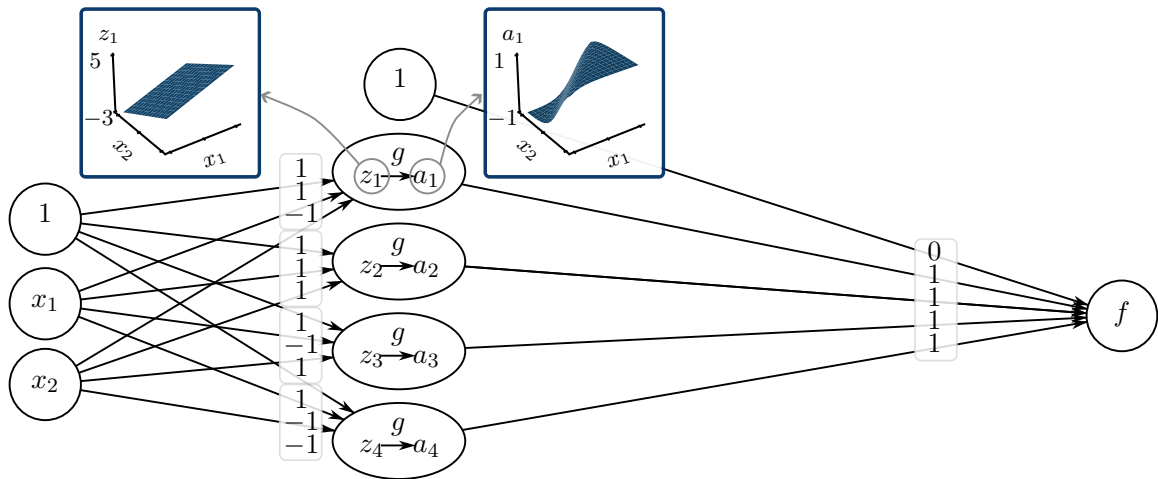
Capacity



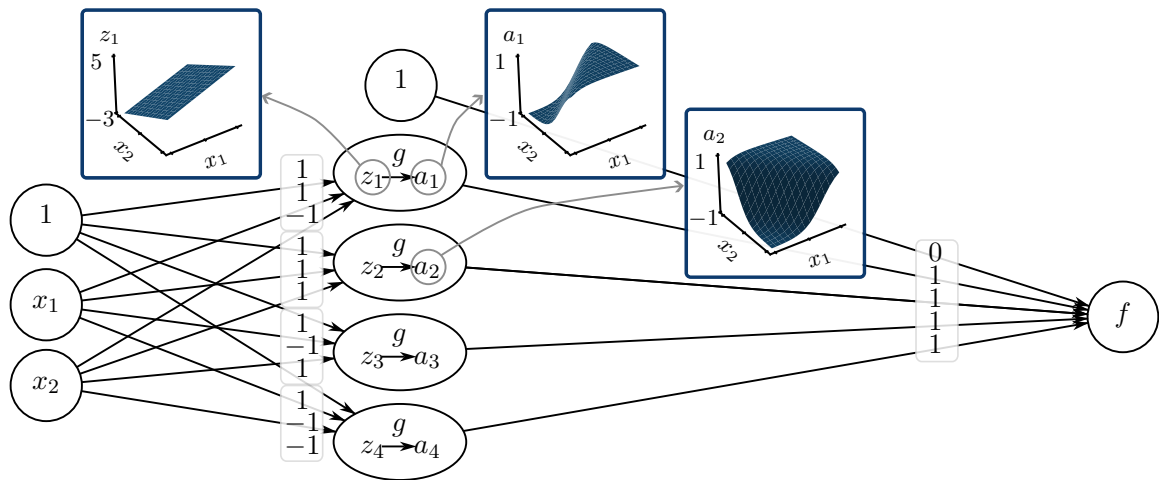
Capacity



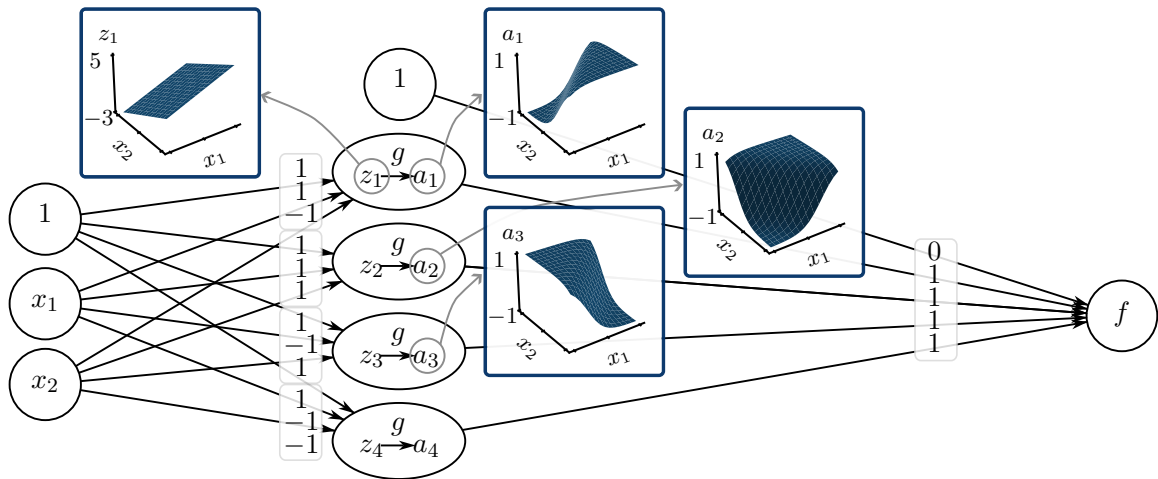
Capacity



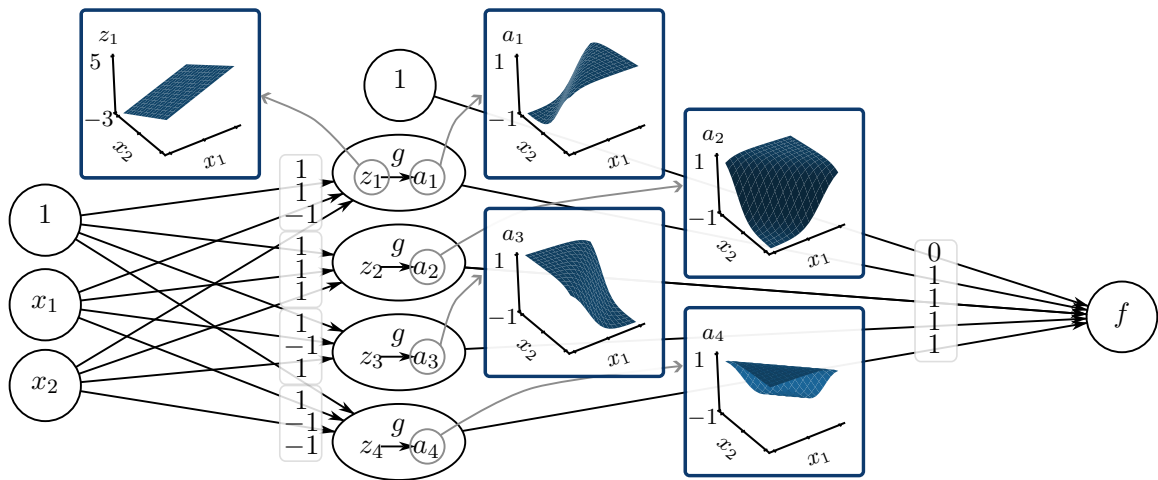
Capacity



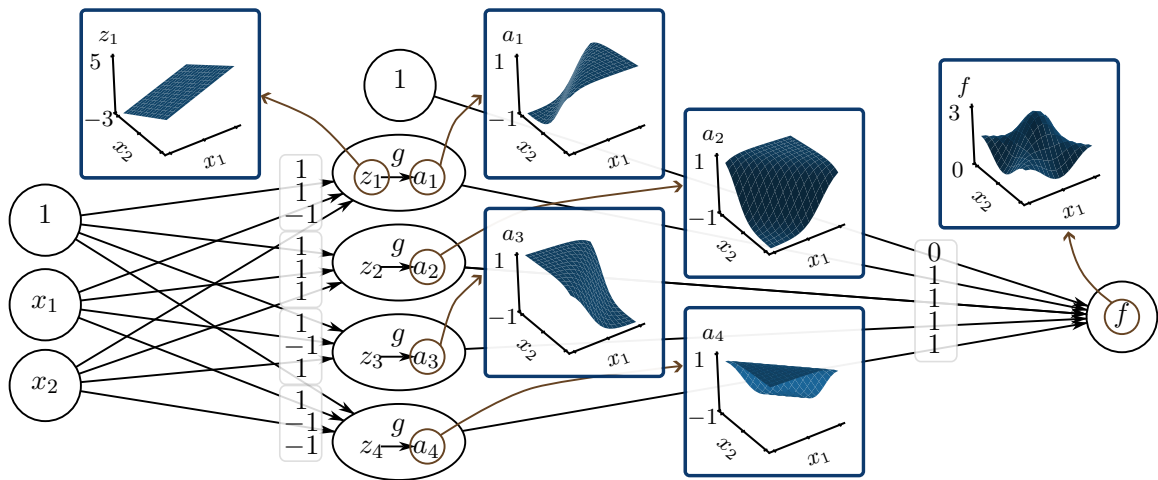
Capacity



Capacity



Capacity



Hidden Layers

How many layers?

- 2 can approximate any function (with enough hidden units)
 - ▶ Like any boolean expression can be written in CNF
 - ▶ ... or any polynomial can be written as sum-of-products
- Historically, this meant not much effort was put into more than 2 layers

Hidden Layers

How many layers?

- 2 can approximate any function (with enough hidden units)
 - ▶ Like any boolean expression can be written in CNF
 - ▶ ... or any polynomial can be written as sum-of-products
- Historically, this meant not much effort was put into more than 2 layers
- But, functions we wish to learn often have internal structure:

$$f(x) = (x - 1)^2 + (x - 1)^4 \quad 4 \text{ ops}$$

$$f(x) = x^4 - 4x^3 + 7x^2 - 6x + 2 \quad 10 \text{ ops}$$

and are simpler when expressed with nested operations

Hidden Layers

How many layers?

- 2 can approximate any function (with enough hidden units)
 - ▶ Like any boolean expression can be written in CNF
 - ▶ ... or any polynomial can be written as sum-of-products
- Historically, this meant not much effort was put into more than 2 layers
- But, functions we wish to learn often have internal structure:

$$f(x) = (x - 1)^2 + (x - 1)^4 \quad 4 \text{ ops}$$

$$f(x) = x^4 - 4x^3 + 7x^2 - 6x + 2 \quad 10 \text{ ops}$$

and are simpler when expressed with nested operations

How many units?

- Too many can cause overfitting
- Use regularization to prevent overfitting
- (so usually add “enough” and use regularization)